

Curso de Docker para Iniciantes

1. O que é Docker e por que ele existe

Imagine que você desenvolveu uma aplicação que funciona perfeitamente no seu computador, mas quando você manda para um colega (ou para o servidor de produção), ela simplesmente não roda. Faltou uma biblioteca, a versão do Python é diferente, uma variável de ambiente não foi configurada... Esse problema é tão comum que virou piada: **"na minha máquina funciona"**.

O Docker resolve exatamente isso. Ele empacota sua aplicação **junto com tudo que ela precisa para rodar** (código, bibliotecas, dependências, configurações) em uma unidade isolada chamada **container**. Esse container roda exatamente igual em qualquer lugar: no seu notebook, no servidor da empresa, ou na nuvem.

O nome "Docker" vem justamente da ideia de **contêineres de navio**: antes deles existirem, carregar navios era caótico, cada tipo de carga era embalada de um jeito diferente. O contêiner padronizado revolucionou o transporte porque qualquer contêiner podia ser carregado, empilhado e transportado da mesma forma, não importa o que tivesse dentro. O Docker faz isso com software.

2. Containers vs Máquinas Virtuais

É comum confundir os dois conceitos:

- **Máquina Virtual (VM)**: simula um computador inteiro, com seu próprio sistema operacional completo. Pesada (gigabytes), demora para iniciar (minutos).
- **Container**: compartilha o sistema operacional da máquina hospedeira, isolando apenas a aplicação e suas dependências. Leve (megabytes), inicia em segundos.

Pense assim: uma VM é como construir uma casa inteira nova para cada morador. Um

container é como um apartamento em um prédio — a estrutura (o "sistema operacional") já existe e é compartilhada, mas cada apartamento é isolado e independente.

3. Conceitos fundamentais

- **Imagem (image)**: um "molde" ou "receita" somente leitura que contém tudo que a aplicação precisa (código, dependências, configurações). É a planta de uma casa.
- **Container**: uma instância em execução de uma imagem. É a casa construída a partir da planta. Você pode ter vários containers rodando a partir da mesma imagem.
- **Dockerfile**: um arquivo de texto com instruções passo a passo para construir uma imagem.
- **Docker Hub / Registry**: um repositório onde imagens ficam armazenadas e podem ser baixadas (parecido com o GitHub, mas para imagens).
- **Volume**: um espaço de armazenamento que persiste dados mesmo depois que o container é removido.

4. Instalando o Docker

Baixe o **Docker Desktop** (Windows/Mac) ou o **Docker Engine** (Linux) em [docker.com](https://www.docker.com/products/docker-desktop/). Depois de instalar, confirme no terminal:

```
```bash
docker --version
```
```

5. Seus primeiros comandos

Vamos rodar seu primeiro container:

```
```bash
```

```
docker run hello-world
```

```
```
```

Esse comando baixa uma imagem simples do Docker Hub e roda um container que só imprime uma mensagem de boas-vindas. Agora os comandos essenciais do dia a dia:

```
```bash
```

```
docker ps # lista containers em execução
```

```
docker ps -a # lista TODOS os containers (parados também)
```

```
docker images # lista imagens baixadas na sua máquina
```

```
docker pull nginx # baixa uma imagem sem rodar
```

```
docker stop <id/nome> # para um container
```

```
docker rm <id/nome> # remove um container parado
```

```
docker rmi <id/nome> # remove uma imagem
```

```
docker logs <id/nome> # mostra os logs de um container
```

```
docker exec -it <id> bash # abre um terminal dentro do container
```

```
```
```

Experimente rodar um servidor web de verdade:

```
```bash
```

```
docker run -d -p 8080:80 nginx
```

```
```
```

- `-d`` roda em segundo plano (detached)

- `-p 8080:80`` mapeia a porta 8080 do seu computador para a porta 80 dentro do container

Depois, acesse `http://localhost:8080`` no navegador e verá a página padrão do Nginx.

6. Criando sua própria imagem: o Dockerfile

Digamos que você tenha uma aplicação Node.js simples. Crie um arquivo chamado `Dockerfile` (sem extensão) na raiz do projeto:

```
```dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```
```

O que cada linha faz:

- `FROM`: define a imagem base (aqui, Node.js 18 na versão leve "alpine")
- `WORKDIR`: define a pasta de trabalho dentro do container
- `COPY`: copia arquivos do seu computador para dentro da imagem
- `RUN`: executa um comando durante a construção da imagem (instalar dependências)
- `EXPOSE`: documenta qual porta a aplicação usa
- `CMD`: comando executado quando o container inicia

Agora construa e rode:

```
```bash
docker build -t minha-app .
docker run -p 3000:3000 minha-app
```
```

`-t minha-app` dá um nome (tag) à sua imagem. O `.` no final diz "use o Dockerfile da pasta atual".

Dica: crie também um arquivo `.dockerignore` (igual ao `.gitignore`) para não copiar `node_modules` e arquivos desnecessários para dentro da imagem.

7. Persistindo dados com Volumes

Por padrão, quando um container é removido, todos os dados dentro dele somem junto. Para bancos de dados ou arquivos que precisam sobreviver, use **volumes**:

```
```bash
docker run -v meus-dados:/app/data minha-app
```
```

Isso cria (ou reutiliza) um volume chamado `meus-dados` e o conecta à pasta `/app/data` dentro do container. Mesmo que o container seja destruído, o volume permanece.

8. Conectando containers com Networks

Containers podem conversar entre si através de redes:

```
```bash
docker network create minha-rede
docker run --network minha-rede --name banco postgres
docker run --network minha-rede --name app minha-app
```
```

Dentro da mesma rede, o container `app` pode acessar o `banco` só usando o nome `banco` como se fosse um endereço.

9. Docker Compose: orquestrando vários containers

Na prática, uma aplicação real geralmente precisa de vários containers (app + banco de dados, por exemplo). Gerenciar isso manualmente é trabalhoso. O ****Docker Compose**** resolve com um arquivo `docker-compose.yml`:

```
```yaml
version: '3.8'
services:
 app:
 build: .
 ports:
 - "3000:3000"
 depends_on:
 - db
 db:
 image: postgres:15
 environment:
 POSTGRES_PASSWORD: senha123
 volumes:
 - dados-postgres:/var/lib/postgresql/data

volumes:
 dados-postgres:
```
```

Com esse arquivo na pasta do projeto, basta rodar:

```
```bash
docker-compose up # sobe tudo (app + banco)
docker-compose down # derruba tudo
```
```

O Compose cria automaticamente uma rede compartilhada entre os serviços, então o ``app`` pode acessar o banco usando ``db`` como endereço.

10. Boas práticas para iniciantes

- Prefira imagens oficiais e leves (com sufixo ``alpine`` quando existir)
- Sempre use um ``.dockerignore``
- Uma responsabilidade por container (não coloque app + banco no mesmo container)
- Evite a tag ``latest`` em produção — use versões específicas (``node:18-alpine``, não só ``node``)
- Não rode processos como usuário ``root`` dentro do container em produção
- Limpe containers e imagens não usados periodicamente com ``docker system prune``

11. Próximos passos

Depois de dominar esse básico, os caminhos naturais são:

- Praticar containerizando um projeto seu do zero
- Aprofundar em Docker Compose com múltiplos serviços
- Estudar registries privados (para guardar suas próprias imagens)
- Só depois disso, explorar orquestração em larga escala com **Kubernetes**

Pratique cada comando digitando você mesmo no terminal — Docker se aprende fazendo.